

LNMIIT Library Management System

User Management Module

Product Requirements Document

Document Version	1.0
Date	24 April 2026
Project	LNMIIT Library Management System
Module	User Management Module
Tech Stack	HTML, CSS, JavaScript, Node.js, Express.js, MongoDB
Team (Group 37)	Aarohi Sinha (24UCS225), Nandindi Sharma (24UCS273), Garv Jain (24UCS270), Kartik Mantri (24UCS246)
Status	Draft — For Review & Viva

1. Executive Summary

The User Management Module is a foundational component of the LNMIIT Library Management System. It provides secure, role-based identity and access management for all system users — including Students, Staff, and Administrators. This document defines the complete product requirements, system architecture, API specifications, database design, and UI/UX flow needed to build and demonstrate this module during the viva.

This module serves as the gateway to the entire library system: every interaction — from borrowing books to managing fines — begins with user authentication through this module.

2. Project Scope & Objectives

2.1 In-Scope Features

- User Registration — New users (students, staff, admin) can be created
- User Login / Authentication — Secure login using JWT tokens
- User Profile Management — View and update user details
- Role-Based Access Control (RBAC) — Different permissions for Student, Staff, Admin
- Update User — Admins/Staff can modify user records
- Retrieve User Details — Fetch individual or all user data
- User Deactivation / Deletion — Admin can deactivate or delete user accounts
- Password Management — Hashed passwords, change password functionality

2.2 Out-of-Scope (Handled by Other Groups)

- Book cataloging and inventory — Group 38 (Book Management)
- Issue & Return tracking — Group 39
- Fine calculation — Group 40
- Book reservation — Group 41

2.3 Goals

- Provide a secure, scalable identity layer for the library system
- Enable clean API endpoints consumable by all other modules
- Demonstrate full CRUD operations on user data during viva
- Follow RESTful API design principles

3. User Roles & Permissions

The system defines three roles, each with distinct access levels:

Permission	Student	Staff	Admin
Register self	✓	✓ (by Admin)	✓ (by Admin)
Login	✓	✓	✓
View own profile	✓	✓	✓
Edit own profile	✓ (limited)	✓	✓
View all users	X	✓	✓
Create any user	X	✓	✓
Update any user	X	✓	✓
Delete / Deactivate user	X	X	✓
Change any password	X	X	✓

4. Tech Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript (ES6+)	User interface — forms, dashboards, profile pages
Backend	Node.js + Express.js	REST API server, business logic, middleware
Database	MongoDB (Mongoose ODM)	Persistent storage for user documents
Authentication	JWT (jsonwebtoken)	Stateless token-based session management
Password Security	bcryptjs	Hashing & salting passwords
Validation	express-validator / Joi	Input validation and sanitization
Environment	dotenv	Manage environment variables securely
API Testing	Postman / Thunder Client	Test and document API endpoints

5. System Architecture

5.1 Architecture Overview

The module follows a classic 3-tier MVC architecture:

Tier	Components	Responsibility
Presentation	HTML, CSS, JS (Frontend)	Render UI, collect form input, call APIs via fetch/axios
Application	Express.js Routes, Controllers, Middleware	Handle HTTP requests, validate input, apply business logic
Data	MongoDB + Mongoose Models	Persist and query user data

5.2 Folder Structure

Path	Description
------	-------------

/frontend/	HTML, CSS, JS files
/backend/	Node.js / Express app root
/backend/models/User.js	Mongoose User schema
/backend/routes/userRoutes.js	Express route definitions
/backend/controllers/userController.js	Business logic functions
/backend/middleware/auth.js	JWT verification middleware
/backend/middleware/role.js	Role-based access control
/backend/config/db.js	MongoDB connection
/backend/.env	Environment variables (secret)
/backend/server.js	App entry point

6. Database Design — MongoDB Schema

6.1 User Collection Schema

Note: The borrowedBooks field is NOT part of the User schema. Book-borrowing references are managed by the Book Management module (Group 38). The User schema only stores identity and access information.

Field	Type	Required	Description
_id	ObjectId	Auto	MongoDB primary key
name	String	Yes	Full name of the user
email	String	Yes (Unique)	User email — used for login
password	String	Yes	bcrypt hashed password (never returned in responses)
role	Enum	Yes	student staff admin
rollNo	String	Conditional	Required for students (e.g. 24UCS225)
phone	String	No	Contact number
address	String	No	Optional postal address
isActive	Boolean	Default: true	Account active/deactivated flag
profilePhoto	String	No	URL or path to profile image
createdAt	Date	Auto	Account creation timestamp (via timestamps)
updatedAt	Date	Auto	Last modification timestamp (via timestamps)

6.2 Mongoose Model Code Snippet

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const UserSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['student', 'staff', 'admin'], default: 'student' },
```

```

rollNo: { type: String },
phone: { type: String },
address: { type: String },
isActive: { type: Boolean, default: true },
profilePhoto: { type: String },
}, { timestamps: true });

// Hash password before saving
UserSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

module.exports = mongoose.model('User', UserSchema);

```

7. API Specification (REST Endpoints)

Base URL: <http://localhost:5000/api/users> | **Auth:** Bearer JWT Token in Authorization header

7.1 Authentication Endpoints

Method	Endpoint	Access	Description
POST	/api/users/register	Public	Register a new user
POST	/api/users/login	Public	Login and receive JWT token
POST	/api/users/logout	Authenticated	Invalidate session / clear token
PUT	/api/users/change-password	Authenticated	Change own password

7.2 User Management Endpoints

Method	Endpoint	Access	Description
GET	/api/users/	Admin/Staff	Retrieve all users (with pagination & filter)
GET	/api/users/:id	Admin/Staff/Self	Retrieve a single user by ID
GET	/api/users/me	Authenticated	Get own profile
PUT	/api/users/:id	Admin/Staff	Update user details
PATCH	/api/users/:id/deactivate	Admin only	Deactivate a user account
DELETE	/api/users/:id	Admin only	Permanently delete a user
GET	/api/users/search?q=	Admin/Staff	Search users by name/email/rollNo

7.3 Sample Request/Response — Register User

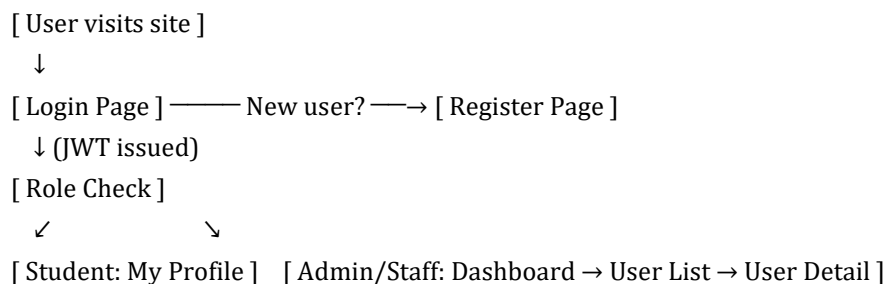
POST /api/users/register — Request Body	Response (201 Created)
---	------------------------

<pre>{ "name": "Aarohi Sinha", "email": "aarohi@lnmiit.ac.in", "password": "SecurePass@123", "role": "student", "rollNo": "24UCS225" }</pre>	<pre>{ "success": true, "message": "User registered", "token": "eyJhbGciOi...", "user": { "_id": "64a...", "name": "Aarohi Sinha", "role": "student" } }</pre>
--	--

8. Frontend Pages & UI Flow

Page / Screen	Route / File	Description
Login Page	/login.html	Email + password form. Returns JWT, redirects to dashboard
Registration Page	/register.html	Form to create a student account. Validates all fields client-side
Admin Dashboard	/admin/dashboard.html	Overview: total users, active/inactive counts, quick actions
User List Page	/admin/users.html	Paginated table of all users. Search, filter by role, deactivate
User Detail Page	/admin/user-detail.html?id=	Full profile view. Edit fields inline. Admin-only delete button
My Profile Page	/profile.html	Student/Staff own profile. Edit name, phone, photo
Change Password	/change-password.html	Old password + new password + confirm. JWT protected

8.1 UI Flow (Textual)



9. Functional Requirements

FR-01: User Registration

- System shall allow new students to self-register with name, email, password, roll number
- Admin/Staff accounts shall only be created by an existing Admin

- Email must be unique; duplicate email returns HTTP 409 Conflict
- Password must be minimum 8 characters with at least 1 number and 1 special character
- Password stored as bcrypt hash (salt rounds = 12) — never plain text

FR-02: User Authentication (Login)

- System shall validate email + password combination
- On success, a signed JWT token is returned (expiry: 24 hours)
- Invalid credentials return HTTP 401 Unauthorized
- Deactivated accounts cannot log in — return HTTP 403 Forbidden

FR-03: Retrieve User Details

- GET /api/users/:id returns full user object (minus password field)
- Admins and Staff can retrieve any user's details
- Students can only retrieve their own profile
- Non-existent IDs return HTTP 404 Not Found

FR-04: Update User

- PUT /api/users/:id allows updating: name, phone, address, role (admin only), isActive
- Email is immutable after registration
- Role changes can only be made by Admin
- Successful update returns updated user object with HTTP 200 OK

FR-05: Delete / Deactivate User

- Soft delete: PATCH /api/users/:id/deactivate sets isActive = false
- Hard delete: DELETE /api/users/:id permanently removes the record (Admin only)
- Cannot delete Admin accounts through the API (safety guard)

10. Non-Functional Requirements

Category	Requirement
Security	All passwords hashed with bcrypt. JWT signed with secret key stored in .env. HTTPS in production.
Performance	API responses under 300ms for standard operations. MongoDB indexed on email and rollNo fields.
Scalability	Stateless JWT auth allows horizontal scaling. MongoDB supports schema evolution.
Availability	Application should be accessible 99% uptime in local/college server environment.
Usability	Forms must show inline validation errors. Mobile-responsive UI built with vanilla HTML/CSS/JS.
Maintainability	MVC pattern enforced. Separate route, controller, model files. Code commented.
Input Validation	All inputs validated server-side using express-validator. Client-side validation is supplementary.
Error Handling	All API errors return standardized JSON: { success: false, message: '...' }

11. Security Considerations

11.1 Authentication Security

- JWT secret stored in .env — never hardcoded
- Token expiry set to 24h; refresh mechanism optional
- HTTP-only cookies can store token to prevent XSS theft

11.2 Authorization

- Middleware checks JWT on every protected route
- Role middleware applied to admin-only routes
- User can only access/modify their own data unless Admin

11.3 Input Security

- Mongoose schema types prevent NoSQL injection
- express-validator sanitizes all input
- Passwords never logged or returned in API responses

12. Integration with Other Modules

The User Management Module exposes its User model and JWT middleware to all other groups in the Library Management System:

Consuming Module	Group	What They Use
Book Management	38	User _id to link book records to borrowers
Issue & Return	39	User _id, role verification, auth middleware
Fine Management	40	User _id, email (for fine notifications)
Reservation	41	User _id, role, isActive check before allowing reservation

Our module provides: auth middleware (middleware/auth.js), role checker (middleware/role.js), and User model (models/User.js) as shared utilities for all consuming modules.

13. Development Milestones

Phase	Task	Deliverables
1	Project Setup	Node.js server, MongoDB connection, folder structure, .env
2	Database Layer	Mongoose User schema, model, pre-save password hook
3	Auth Endpoints	Register, Login, JWT generation, protected route middleware

4	CRUD Endpoints	Get all, Get by ID, Update, Delete, Deactivate endpoints
5	Frontend	Login, Register, Dashboard, Profile, User List pages in HTML/CSS/JS
6	Integration	Connect frontend to backend, handle CORS, test all flows
7	Testing & Viva Prep	Postman collection, test all edge cases, demo script

14. Viva Preparation Checklist

Concepts to Know

- What is REST API? Why do we use it?
- What is JWT? How does it work? Why not sessions?
- Why is bcrypt used? What is salting?
- What is middleware in Express.js?
- What is RBAC (Role-Based Access Control)?
- Why MongoDB over SQL for this project?
- What is Mongoose? What is a Schema?
- What is the difference between PUT and PATCH?
- How does CORS work and why do we need it?

Live Demo Script

- Show folder structure and explain MVC architecture
- Open MongoDB Compass / Atlas — show empty users collection
- Register a new student via Postman — show hashed password in DB
- Login — show JWT token returned
- Use token to access GET /api/users/ (protected)
- Update user details — show change in DB
- Deactivate user — try logging in again (should fail)
- Show frontend: Login → Dashboard → User List → Profile edit

15. Team Responsibilities

Roll No.	Name	Group	Responsibility
24UCS225	Aarohi Sinha	37	Project Lead, API Design, Register & Login endpoints
24UCS273	Nandindi Sharma	37	Frontend: Login, Register, Profile pages
24UCS270	Garv Jain	37	Update User endpoint, Role middleware, JWT implementation
24UCS246	Kartik Mantri	37	Retrieve User Details endpoint, Search & Filter API

End of Document — LNMIIT Library Management System / User Management Module PRD

Confidential — Group 37 Only